

# INTRODUCTION TO PYTHON

# OBJECTS

---

- Programs manipulate **data objects**
- Objects have a **type** that defines the kinds of things programs can do to them
  - A car is an **object that can** start, accelerate, locks the car doors
  - A car is an object that cannot fly, teleport, time travel
- Objects are:
  - Scalar (cannot be subdivided)
  - Non-Scalar (have internal structure that can be accessed)
    - A date is an **object that has** a year, month, and day

# SCALAR OBJECTS

---

- **int** – represent **integers**, ex. 7
- **float** – represent real numbers, ex. 4.29
- **bool** – represent **Booleans** values True and False
- **NonType** – **special** and has one value, None
- Use **type()** to see the type of an object

# TYPE CONVERSIONS

---

- can **convert objects of one type to another**
- **float**(3) converts integer 3 to float 3.0
- **int** (5.7) converts float 5.7 to int 5

# OPERATORS ON ints and floats

---

- $i + j$   the **sum** 
  - $i - j$   the **difference** 
  - $i * j$   the **product** 
- if both are ints, result is int  
if either or both are floats, result is float
- 
- $i / j$   **division**  **result is float**
- 
- $i // j$   **floor division**
  - $i \% j$   the **remainder** when  $i$  is divided by  $j$
  - $i ** j$    $i$  to the **power** of  $j$

# STRINGS

---

- Letters, special characters (#, @, ...), spaces, digits
- Cannot change elements values, **immutable**
- Enclose in **quotation marks** or **single quotes**

```
string_example = "hello there"
```

```
string_example = 'hello there'
```

```
string_example = ""
```

*empty string*

# OPERATIONS ON str

---

- i and j are string variables
  - `i + j`       `concatenate` strings
  - `i[ 0 ]`       `first character` of the string
  - `i[ 0:2 ]`       `slice` a string, first two characters
  - `i[ -1 ]`       `last character` of the string
  - `i*2`       `multiplication`, string repetition
  - `'substring' in i`       `in`, check whether a substring appears in a string
- 
- i and j are float variables, ex `i = 3.5, j = 7.2`
  - `'value1 = {0}, value2 = {1}'.format(i, j)`       `str.format`, format strings
  - `f'value1 = {i}, value2 = {j}'`       `f-strings`, format strings

# COMPARSION OPERATORS ON int, float, string

---

- i and j are variable names
- comparisons below evaluate to a Boolean

`i > j`

`i >= j`

`i < j`

`i <= j`

`i == j`            **equality** test, True if i is the same as j

`i != j`            **inequality** test, True if i not the same as j

# CONTROL FLOW - BRANCHING

---

**if** <condition>:

    <expression>

    <expression>

    <expression>

...

**if** <condition>:

    <expression>

    <expression>

    <expression>

...

**else:**

    <expression>

    <expression>

    <expression>

...

- <condition> has a value *True* or *False*

# IDENTATION

---

- Matters in Python
- How you denote block of code

```
x = 5
```

```
y = 7
```

```
if x < y:
```

```
    print("x is smaller")
```

```
else:
```

```
    print("y is smaller")
```

```
print("thanks!")
```

# CONTROL FLOWS: while LOOPS

---

**while** <condition>:

    <expression>

    <expression>

    <expression>

    ...

- <condition> evaluates to a Boolean
- If <condition> is True, do all the steps inside while code block
- Check <condition> again
- Repeat until <condition> Is False

# CONTROL FLOWS: while VS for

---

- Iterate through numbers in a sequence
- **More complicated** with **while** loop:

```
n = 0
```

```
while n < 5:
```

```
    print(n)
```

```
    n = n + 1
```

- **Shortcut** with **for** loop:

```
for n in range(5):
```

```
    print(n)
```

# CONTROL FLOWS: for LOOPS

---

**for** <variable> **in range**(some\_num):

    <expression>

    <expression>

    <expression>

...

- Each time through the loop, <variable> takes a value
- First time, <variable> starts at the smallest value
- Next time, <variable> gets the previous value + 1
- Repeat until <variable> is smaller than some\_num

# range (start, stop, step)

---

- Default values are start = 0 and step = 1
- Loop until value is stop - 1

```
my_sum = 0
for i in range(7, 10):
    my_sum += i
print(my_sum)
```

```
my_sum = 0
for i in range(5, 11, 2):
    my_sum += i
print(my_sum)
```

# TUPLES

- An **ordered sequence of elements**,
- Can mix element types
- Cannot change elements values, **immutable**
- Represented with parentheses

*remember  
strings?*

*empty  
tuple*

```
tuple_example = ()
```

```
tuple_example = (2, "uab", 3.6)
```

|                                       |                                                                                      |                                        |
|---------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------|
| <code>tuple_example[0]</code>         |  | evaluates to 2                         |
| <code>(2, "uab", 3.6) + (5, 6)</code> |  | evaluates to (2, "uab", 3.6, 5, 6)     |
| <code>tuple_example[1:3]</code>       |  | slice tuple, evaluates to ("uab", 3.6) |
| <code>len(tuple_example)</code>       |  | evaluates to 3                         |
| <code>tuple_example[1] = 4</code>     |  | gives error, can't modify object       |

# LISTS

---

- **Ordered sequence** of information, accessible by index
- A list is denoted by square brackets, []
- A list contains **elements**
  - Usually homogeneous, ex all integers
  - Can contain mixed types
- List elements can be changed, so a list is **mutable**

# INDICES AND ORDERING

---

```
list_example = []
```

*empty list*

```
list_example = [ 2, 'a' , 4 , [ 1, 2 ] ]
```

|                           |   |                                    |
|---------------------------|---|------------------------------------|
| <b>len</b> (list_example) | → | evaluates to 4                     |
| list_example[0]           | → | evaluates to 2                     |
| list_example[2] + 1       | → | evaluates to 5                     |
| list_example[3]           | → | evaluates to [1, 2], another list! |
| list_example[4]           | → | gives an error, out of range       |

# CHANGING ELEMENTS

---

- List are **mutable**
- Assigning to an element at an index changes the value

```
list_example = [ 2, 1, 3 ]
```

```
list_example [1] = 5
```

- list\_example **is now** [ 2, 5, 3 ], note this is the **same object** list\_example
- In these cases, Python Tutor is your friend!!!

# ITERATING OVER A LIST

---

- Compute the sum of elements of a list
- Common patterns, iterate over list elements

```
total = 0
```

```
for i in range(len(L)):
```

```
    total += L[i]
```

```
print(total)
```

```
total = 0
```

```
for i in L:
```

```
    total += i
```

```
print(total)
```

- list elements are indexed from 0 to **len(L)** - 1
- **range(n)** goes from 0 to n-1

# DICTIONARIES

---

- We want to store names and ages of people
- We can store it using separate list for every info

```
names = ['Ana', 'John', 'Denise', 'Katy']
```

```
ages = [21, 33, 26, 30]
```

- A **separate list** for each item
- Each list must have the **same length**
- Info stored across lists at **same index**

```
ages = {'Ana': 21, 'John': 33, 'Denise': 26, 'Katy': 30}
```

# A PYTHON DICTIONARY

---

- Store pairs of data
  - Key
  - Value

ages = {} *empty dictionary*

ages = {'Ana': 21, 'John': 33, 'Denise': 26, 'Katy': 30}

↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑  
key1 val1 key2 val2 key3 val3 key4 val4

# DICTIONARY LOOKUP

---

- Similar to indexing into a list
- Looks up the **key**
- Returns the **value** associated with the key
- If key isn't found, get an error

```
ages = { 'Ana' : 21, 'John' : 33, 'Denise': 26, 'Katy': 30 }
```

```
ages['John']  —————>  33
```

```
ages['Peter'] —————>  gives a KeyError
```

# DICTIONARY OPERATIONS

---

```
ages = { 'Ana' : 21, 'John' : 33, 'Denise': 26, 'Katy': 30 }
```

- **Add** an entry

```
ages['Peter'] = 25
```

- **Test** if key in dictionary

```
'Ana' in ages    —————>  returns True
```

```
'Daniel' in ages —————>  returns False
```

- **Delete** an entry

```
del(ages['John'])
```

# DICTIONARY OPERATIONS

---

ages = { 'Ana' : 21, 'John' : 33, 'Denise': 26, 'Katy': 30 }

- Get **all keys**

ages.keys()       $\longrightarrow$       returns ['Ana', 'John', 'Denise', 'Katy']

- Get **all values**

ages.values()       $\longrightarrow$       returns [21, 33, 26, 30]

- Get **all pairs key/value**

ages.items()       $\longrightarrow$       returns [('Ana', 21), ('John', 33), ('Denise', 26), ('Katy', 30)]

# DICTIONARY KEYS and VALUES

---

- Keys
  - Must be **unique**
  - Types: int, string
- Values
  - Can be **duplicates**
  - Dictionary values can be lists, even other dictionaries!

# Thank you!!

---